

Pakiety, moduły i testowanie za pomocą pytest.

Utworzymy prosty projekt, składający się z pliku `kalkulator.py` oraz plików testowych:

```
projekt/
├── src/
│   ├── __init__.py
│   └── kalkulator.py
└── tests/
    ├── __init__.py
    ├── test_kalkulator.py
    └── nowy/
        ├── __init__.py
        └── kalkulator_test.py
```

```
# src/__init__.py
# może być pusty
print("Importowanie modulu: " + __name__)
```

```
# src/kalkulator.py
def dodaj(a, b):
    return a + b
```

Spróbujmy interaktywnie (dwa przykłady):

```
>>> import src.kalkulator
Importowanie modulu: src
>>> import src.kalkulator # już zbuforowany
>>> src.kalkulator.dodaj(2,3)
5
```

Import modułu – co wtedy robi Python?

a) przeszukuje ścieżki zapisane w `sys.path`, które obejmują m.in. katalog, w którym został uruchomiony skrypt, katalogi zainstalowanych modułów oraz ścieżki zdefiniowane przez zmienną środowiskową `PYTHONPATH`.
b) Python szuka pliku `.py` lub katalogu o nazwie zgodnej z tym, co próbuje się zaimportować. Na przykład przy `import src.kalkulator` sprawdzi, czy w którejś z lokalizacji z `sys.path` znajduje się plik `src/kalkulator.py` lub katalog `src`, a w nim plik `kalkulator.py`.
c) jeśli katalog zawiera plik `__init__.py`, Python traktuje go jako zwykły pakiet. Katalog, który nie ma pliku `__init__.py`, może być traktowany jako pakiet przestrzenny (namespace package) – wciąż można z niego importować moduły, o ile jego położenie znajduje się na `sys.path`.

W Pythonie pakiet to katalog, który może, ale nie musi, zawierać plik `__init__.py`. Jeśli plik `__init__.py` jest obecny, pakiet może wykonywać kod inicjalizacyjny i ma pełną kontrolę nad swoim zachowaniem. Jeśli `__init__.py` brakuje, katalog jest traktowany jako pakiet przestrzenny (namespace package) – nadal można importować moduły z takiego katalogu, pod warunkiem że jego lokalizacja znajduje się na `sys.path`. Moduł to nadal po prostu pojedynczy plik `.py`.

```
>>> from src.kalkulator import dodaj
Importowanie modulu: src
>>> dodaj(2,3)
5
```

```
from src.kalkulator import * # bez kontroli co importujemy
```

Sprawdzenie, czy w (słowniku) załadowanych `sys.modules` jest nasz moduł:

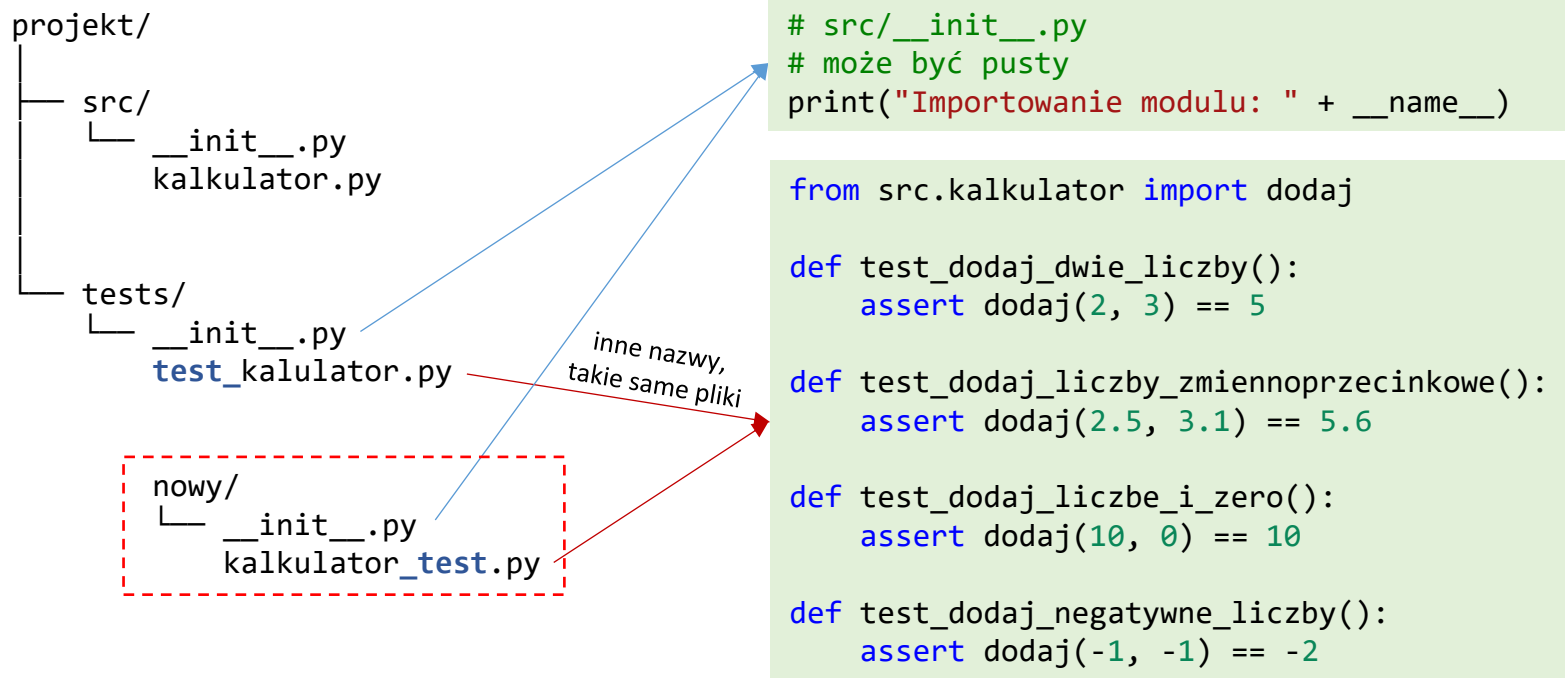
```
>>> import sys
>>> if 'src.kalkulator' in sys.modules:
...     print("tak, jest")
...
tak, jest
```

Jeśli chcemy raz jeszcze załadować zbuforowane moduły:

```
>>> import src.kalkulator
Importowanie modulu: src
>>> import importlib
>>> importlib.reload(src) # tylko src, więc widac print()
Importowanie modulu: src
<module 'src' from 'C:\\tmp\\PYTHON\\2024\\GR4\\projekt\\src\\__init__.py'>
>>> importlib.reload(src.kalkulator)
<module 'src.kalkulator' from 'C:\\tmp\\PYTHON\\2024\\GR4\\projekt\\src\\kalkulator.py'>
```

Pakiety, moduły i testowanie za pomocą **pytest**. Zainstalować: `pip install pytest`

Utworzymy prosty projekt, składający się z pliku `kalkulator.py` oraz plików testowych `test_kalkulator.py` itp.:



Gdzie pytest szuka testów?

- szuka plików, które spełniają poniższe kryteria nazewnictwa:
 - pliki zaczynające się od `test_` (np. `test_kalkulator.py`).
 - pliki kończące się na `_test.py` (np. `kalkulator_test.py`).
 - pliki te mogą znajdować się w katalogu, z którego uruchamiasz pytest, lub w dowolnym podkatalogu w strukturze projektu (pamiętajmy o `__init__.py`)
- funkcje testujące – w plikach testowych pytest szuka funkcji, które:
 - mają nazwy zaczynające się od `test_` (np. `test_dodaj()`).
 - pytest automatycznie uruchomi te funkcje i sprawdzi, czy asercje (`assert`) w nich zawarte są poprawne.
- klasy testowe – pytest szuka również klas, które:
 - mają nazwy zaczynające się od `Test` (np. `TestKalkulator`). Zawierają metody testujące, które zaczynają się od `test_`.

```

PS C:\tmp\PYTHON\2024\GR4\projekt> pytest
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-8.2.0, pluggy-1.5.0
rootdir: C:\tmp\PYTHON\2024\GR4\projekt
plugins: anyio-4.2.0
collected 4 items

tests\test_kalkulator.py .... [100%]

===== 4 passed in 0.04s =====
  
```

ZADANIE

- sprawdź wywołanie **pytest** z opcją `-q`, `-s`, `-v` (można `-sv`)
- dodaj podkatalog nowy z zawartością (jak w ramce)
- dodaj w pliku `kalkulator.py` np. funkcję `test_()` i sprawdź, że nie będzie wywołana do testów (bo plik nie spełnia kryteriów nazwy), można jednak pytest wywołać bezpośrednio z plikiem `src/kalkulator.py` (podejście niezalecane)

Wracamy do poprzedniego programu, zapiszemy go jak niżej, w pliku `main.py`

```
def przetworz_wejscie(wejscie):
    wejscie = wejscie.strip().replace(",,", ",")
    lista_slow = wejscie.split(",")
    lista_slow = [slovo.strip().lower() for slovo in lista_slow if slovo.strip()]
    return lista_slow, len(lista_slow)

def main():
    wejscie = input("Podaj kilka slow oddzielonych przecinkami: ")
    lista_slow, liczba_slow = przetworz_wejscie(wejscie)
    print("Lista slow:", lista_slow, "#", liczba_slow)

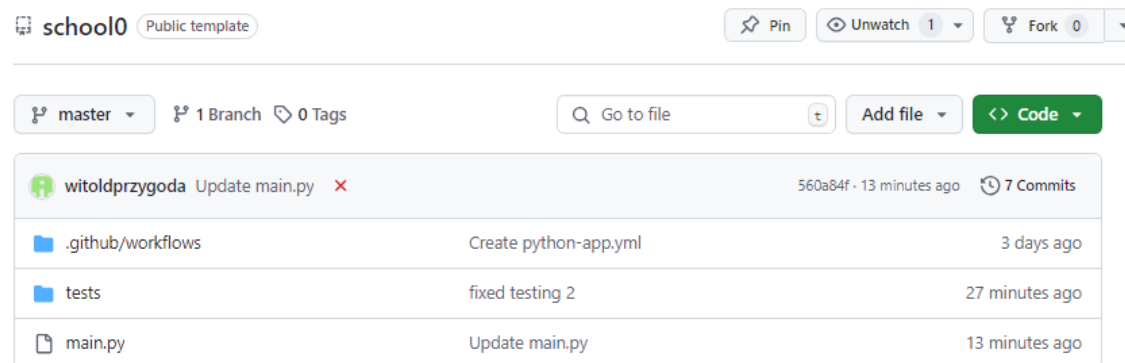
if __name__ == "__main__":
    main()
```

ZADANIE

Dla powyższego programu napisz kilka testów, czyli funkcji zaczynających się od `test_`, zapisz w tym samym katalogu, w pliku `test_main.py` oczywiście uruchom pytest

```
def test_wzorzec():
    wejscie = "" # tutaj wpisz wejscie
    expected_lista = [] # tutaj wpisz oczekiwana liste
    expected_liczba = # tutaj wpisz oczekiwana liczbe
    wynik_lista, wynik_liczba = przetworz_wejscie(wejscie)
    assert wynik_lista == expected_lista
    assert wynik_liczba == expected_liczba
```

Przećwiczmy umieszczanie swojego rozwiązania w ramach GitHub Classroom na przykładzie poprzedniego kodu!



pierwotne repozytorium, będące bazą dla danego zestawu zadań, wraz ze zdefiniowanymi testami

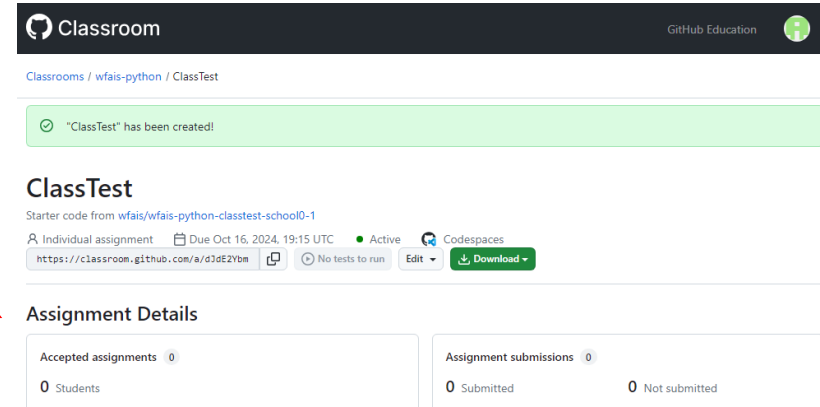
wfais-python

Accept the assignment —
ClassTest

Once you accept this assignment, you will be granted access to the `classtest-pythonuj` repository in the `wfais` organization on GitHub.

Accept this assignment

na swoim koncie GitHub należy najpierw autoryzować Classroom, a potem zaakceptować dane zadanie



zestaw zadań utworzony w Classroom

⇒ gdy się pojawi, na Pegazie będzie oprócz treści zadań (dla wygody – bo będą też w README), będzie podany **link** aby zaakceptować



You're ready to go!

You accepted the assignment, **ClassTest**.

Your assignment repository has been created:

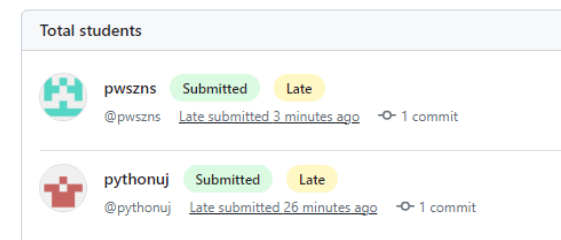
<https://github.com/wfais/classtest-pythonuj>

We've configured the repository associated with this assignment.

Open in GitHub Codespaces

Your assignment was due on Oct 16, 2024, 19:15 UTC

utworzona zostaje kopia repozytorium, celem umieszczenia w niej swoich rozwiązań zadań, po czym push – przećwiczmy to na przykładzie!



 ZADANIE

Napisz program, który wczytuje liczby zmiennoprzecinkowe oddzielone spacjami i konwertuje je na różne typy: float, decimal.Decimal, numpy.float32 i numpy.float64, a następnie sumuje te liczby dla każdego z typów. Na końcu program wypisuje wyniki.

```
from decimal import Decimal
import numpy as np

def main(liczby_input):

    liczby_str = # wczytaj do listy elementy z liczby_input

    # Konwersja do różnych typów
    liczby_float = # [KONWERSJA do float PĘTLA PO liczby_str]
    liczby_decimal = # podobnie
    liczby_numpy_float32 = # podobnie
    liczby_numpy_float64 = # podobnie

    # Sumowanie liczb
    suma_float = # policz sumę funkcją sum
    suma_decimal = # podobnie
    suma_numpy_float32 = # podobnie
    suma_numpy_float64 = # podobnie

    # Zwracamy wyniki
    return suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64

if __name__ == "__main__":
    # Wczytanie liczb od użytkownika przez input()
    liczby_input = input("Podaj liczby zmiennoprzecinkowe oddzielone spacjami: ")

    # Wywołanie funkcji main() z przekazaniem ciągiem liczb
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)

    # Wyświetlenie wyników
    print(f"Suma dla float: {suma_float}")
    print(f"Suma dla decimal: {suma_decimal}")
    print(f"Suma dla numpy.float32: {suma_numpy_float32}")
    print(f"Suma dla numpy.float64: {suma_numpy_float64}")
```

Komentarz do formatowania

F-string (format string) to sposób formatowania łańcuchów znaków w Pythonie (od wersji 3.6). Pozwala na wstawianie wartości zmiennych lub wyrażeń bezpośrednio do łańcucha tekstowego. F-stringi są definiowane przez dodanie litery **f** przed cudzysłowem otwierającym łańcuch. Wewnątrz f-stringa możemy używać nawiasów klamrowych {}, w których umieszczamy zmienne lub wyrażenia, które chcemy wstawić do tekstu. Przykład:

```
name = "Adam"
age = 21
message = f"My name is {name} and I am {age} years old."
```

Wewnątrz nawiasów {} można wykonywać dowolne wyrażenia, np. działania arytmetyczne.

```
result = f"2 + 2 = {2 + 2}"
```

Można precyzyjnie formatować wartości, np. zaokrąglanie liczb zmiennoprzecinkowych.

```
value = 10.12345 print(f"Zaokrąglona wartość: {value:.2f}") # Wyjście: Zaokrąglona wartość: 10.12
```

 ZADANIE

Napisz w osobnym pliku ze 2 funkcje wykonujące test programu. Uruchom za pomocą pytest.

 ZADANIE

Napisz program, który wczytuje liczby zmiennoprzecinkowe oddzielone spacjami i konwertuje je na różne typy: float, decimal.Decimal, numpy.float32 i numpy.float64, a następnie sumuje te liczby dla każdego z typów. Na końcu program wypisuje wyniki.

```
from decimal import Decimal
import numpy as np
```

program, plik main.py

```
def main(liczby_input):
```

```
    liczby_str = liczby_input.split()
```

```
    # Konwersja do różnych typów
```

```
    liczby_float = [float(liczba) for liczba in liczby_str]
```

```
    liczby_decimal = [Decimal(liczba) for liczba in liczby_str]
```

```
    liczby_numpy_float32 = [np.float32(liczba) for liczba in liczby_str]
```

```
    liczby_numpy_float64 = [np.float64(liczba) for liczba in liczby_str]
```

```
    # Sumowanie liczb
```

```
    suma_float = sum(liczby_float)
```

```
    suma_decimal = sum(liczby_decimal)
```

```
    suma_numpy_float32 = sum(liczby_numpy_float32)
```

```
    suma_numpy_float64 = sum(liczby_numpy_float64)
```

```
    # Zwracamy wyniki
```

```
    return suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64
```

```
if __name__ == "__main__":
```

```
    # Wczytanie liczb od użytkownika przez input()
```

```
    liczby_input = input("Podaj liczby zmiennoprzecinkowe oddzielone spacjami: ")
```

```
    # Wywołanie funkcji main() z przekazaniem ciągiem liczb
```

```
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)
```

```
    # Wyświetlenie wyników
```

```
    print(f"Suma dla float: {suma_float}")
```

```
    print(f"Suma dla decimal: {suma_decimal}")
```

```
    print(f"Suma dla numpy.float32: {suma_numpy_float32}")
```

```
    print(f"Suma dla numpy.float64: {suma_numpy_float64}")
```

test, plik test_main.py

```
from decimal import Decimal
```

```
import numpy as np
```

```
from main import main # Importujemy main z pliku main.py
```

```
def test_suma_float():
```

```
    liczby_input = "0.1 0.2 0.3"
```

```
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)
```

```
    assert round(suma_float, 6) == 0.6 # Zaokrąglamy do 6 miejsc
```

```
def test_suma_decimal():
```

```
    liczby_input = "0.1 0.2 0.3"
```

```
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)
```

```
    assert suma_decimal == Decimal('0.6')
```

```
def test_suma_numpy_float32():
```

```
    liczby_input = "0.1 0.2 0.3"
```

```
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)
```

```
    assert np.isclose(suma_numpy_float32, np.float32(0.6), atol=1e-6)
```

```
def test_suma_numpy_float64():
```

```
    liczby_input = "0.1 0.2 0.3"
```

```
    suma_float, suma_decimal, suma_numpy_float32, suma_numpy_float64 = main(liczby_input)
```

```
    assert np.isclose(suma_numpy_float64, np.float64(0.6), atol=1e-9)
```

```
PS C:\tmp\PYTHON\2024\GR4\floats> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-8.2.0, pluggy-1.5.0 -- C:\Python\python.exe
cachedir: .pytest_cache
rootdir: C:\tmp\PYTHON\2024\GR4\floats
plugins: anyio-4.2.0
collected 4 items

test_main.py::test_suma_float PASSED [ 25%]
test_main.py::test_suma_decimal PASSED [ 50%]
test_main.py::test_suma_numpy_float32 PASSED [ 75%]
test_main.py::test_suma_numpy_float64 PASSED [100%]

===== 4 passed in 0.28s =====
```


Kilka testów na temat kopiowania (płytkiego, częściowo głębokiego, głębokiego) – [proszę to przerobić](#)

ZADANIE

Utwórz listę **moja_lista**, wypełnij ją jakimiś elementami, na przykład liczbami całkowitymi. Wykonaj kolejno:

- przypisanie do innej nazwy (1)
- przypisanie z `moja_lista[:]` (2)
- dodaj do `moja_lista` jakąś inną, zagnieżdżoną listę
- ponownie przypisanie do innej nazwy (3)
- zaimportuj moduł `copy` i zrób kopię z `moja_lista` za pomocą `copy` (4)
- podobnie, ale zrób `deepcopy` (5)
- testy: zmień jeden element w zagnieżdżonej liście (1) oraz (2) i sprawdź oryginał
- testy: zmień jeden element w zagnieżdżonej liście (3, 4, 5) i sprawdź oryginał

Na kolejnych slajdach są dwa zadania. Służą za przykład do zapoznania się z zasadami pracy na GitHub Classroom. Zadania można oczywiście rozwiązać niezależnie, ale cel jest taki, żeby kompleksowo przećwiczyć procedury od zapisania się do danego zestawu (ten ćwiczebny nazywa się "Zestaw 0"), poprzez pracę z plikami z repozytorium, testowanie, aż po wysłanie (wypchnięcie) rozwiązań do repozytorium, gdzie zostaną obejrzone i ocenione. Ocena to punktacja, która będzie wpisywana w systemie Pegaz – odrębnie od GitHub i ewentualnej roboczej punktacji zdefiniowanej w ramach danego Assignment.

Dokładny opis pracy z GitHub Classroom znajduje się w osobnym pliku! Bardzo proszę to przećwiczyć!

 ZADANIE

Mamy trzy liczby naturalne, x , y , z reprezentujące wymiary prostopadłościanu, oraz pewną liczbę naturalną n . Wypisz listę wszystkich możliwych współrzędnych (i, j, k) na trójwymiarowej siatce, gdzie $i + j + k$ nie jest równe n . Warunki:

$0 \leq i \leq x$, $0 \leq j \leq y$, $0 \leq k \leq z$. Rozwiązanie zapisz w postaci list składanych (list comprehension).

Przykład. Niech $x = 1$, $y = 1$, $z = 2$, $n = 3$. Lista wszystkich permutacji trójek $[i, j, k]$ w tym przykładzie: $[[0,0,0], [0,0,1], [0,0,2], [0,1,0], [0,1,1], [0,1,2], [1,0,0], [1,0,1], [1,0,2], [1,1,0], [1,1,1], [1,1,2]]$. Elementy, które nie sumują się do 3 to: $[[0,0,0], [0,0,1], [0,0,2], [0,1,0], [0,1,1], [1,0,0], [1,0,1], [1,1,0], [1,1,2]]$. Parametry x , y , z , n wczytać na początku za pomocą funkcji `input()`.

wejście: `1 1 2 3` wyjście: `[[0, 0, 0], [0, 0, 1], [0, 0, 2], [0, 1, 0], [0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 2]]`

Wymagania formalne Użyć plik `ZADANIE1/zadanie1.py` z repozytorium GitHub Classroom do uzupełnienia swoim kodem. Nie zmieniać funkcji `permutacja(x, y, z, n)`, która wywoływana jest w testach.

```
def permutacja(x, y, z, n):
    pass # return lista list

if __name__ == '__main__':
    x, y, z, n = # wczytaj dane
    lista = permutacja(x, y, z, n)
    print(lista)
```

Co teraz zrobić?

- zapisać się do "Zestaw 0" – link na stronie oraz w dokumencie PDF `github_classroom.pdf`
- edytować kod zadania albo w GitHub Codespaces, albo lepiej zrobić klon repozytorium na swój komputer, po czym pracować z kodem źródłowym, uruchamiać testy
- na koniec wysłać rozwiązania do repozytorium zdalnego

 ZADANIE

Napisać funkcję `odwracanie(L, left, right)` odwracającą kolejność elementów na liście od numeru `left` do `right` włącznie. Lista jest modyfikowana w miejscu (in place). Rozważyć wersję iteracyjną i rekurencyjną. Jest to jedno z zadań ze strony z materiałami wykładowymi, zad. 4.5 ➔ <https://ufkapano.github.io/algorytmy/lekcja04/zadania.html>

wejście: `[1, 2, 3, 4, 5, 6]` wyjście dla wywołania z `left 1, right 4`: `[1, 5, 4, 3, 2, 6]`

Wymagania formalne Użyć plik `ZADANIE2/zadanie2.py` z repozytorium GitHub Classroom do uzupełnienia swoim kodem. Nie zmieniać nazw i argumentów funkcji: `odwracanie_iteracyjnie(L, left, right)` i `odwracanie_rekurencyjnie(L, left, right)`, które wywoływane są w testach.

```
def odwracanie_iteracyjnie(L, left, right):
    # twój kod
    return L
```

```
def odwracanie_rekurencyjnie(L, left, right):
    # twój kod
    return L
```

Co teraz zrobić?

- to samo, co dla ZADANIE1
- zapisać się do "Zestaw 0" – link na stronie oraz w dokumencie PDF `github_classroom.pdf`
- edytować kod zadania albo w GitHub Codespaces, albo lepiej zrobić klon repozytorium na swój komputer, po czym pracować z kodem źródłowym, uruchamiać testy
- na koniec wysłać rozwiązania do repozytorium zdalnego

Uwaga: należy rozważyć również indeksy ujemne, zgodnie z regułami indeksowania wielkości sekwencyjnych! Przygotowane testy mogą zaskoczyć, napisanie całkiem poprawnej wersji będzie wymagało trochę wysiłku.

1	2	3	4	5	6
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1